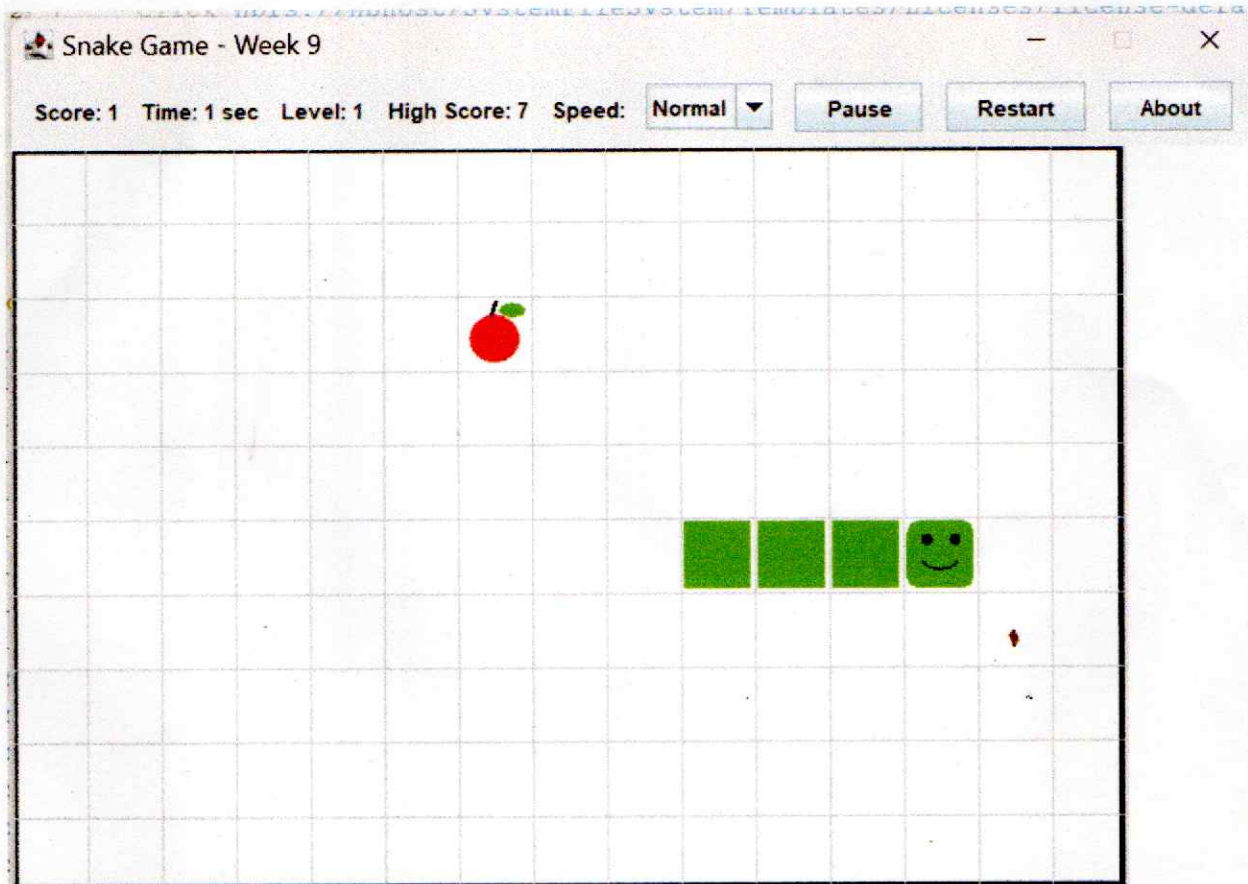


1 Introduction

SnakeGame is a Java-based Snake Game project developed step by step from Week 1 to Week 10. The project started with planning and requirement analysis, followed by software design and user interface development. Then the game board, snake, food, movement, collision detection, score, time, speed control, restart, pause/resume, game levels, high-score system, and winning features were developed gradually.

In the final stage, **Week 10 focuses on complete testing, debugging, documentation, user manual preparation, final presentation, and Version 1.0 release.** The main goal of the project is to develop a complete and playable Snake Game application and prepare it as a properly tested and documented final project.



2 Project Objectives

The main objectives of the **SnakeGame** project are:

1. To develop a complete Snake Game application using **Java and Java Swing**.
2. To create a simple and user-friendly **graphical user interface**.
3. To implement basic snake movement and **keyboard control**.
4. To implement **food collection** and snake growth.

5. To detect **wall and body collisions** and handle the Game Over condition.
6. To implement game features such as **score, time, speed control, restart, pause/resume, and different levels**.
7. To implement a **high-score system** using file storage.
8. To implement **target-score based winning** and appropriate Success/Game Over messages.
9. To test the complete application and **fix bugs and errors**.
10. To prepare the **documentation, user manual, screenshots, final presentation, and Version 1.0 release**.

3 Development Methodology

The **SnakeGame** project was developed using an **incremental and step-by-step development approach**. The project was divided into different weekly stages, and each week added or improved specific features of the application.

- **Week 1–2:** Project planning, requirement analysis, and software design.
- **Week 3:** User interface and navigation system development.
- **Week 4:** Game board, snake, and food development.
- **Week 5–6:** Snake movement, food collection, and collision detection.
- **Week 7:** Score, time, speed control, restart, and interface improvements.
- **Week 8:** Pause/resume, different levels, high-score storage, and performance improvement.
- **Week 9:** Winning/target-score system, Game Over/Success messages, UI improvement, and About section.
- **Week 10:** Complete testing, debugging, documentation, final presentation, and Version 1.0 release.

4 Software and Hardware Requirements

1. Hardware Requirements

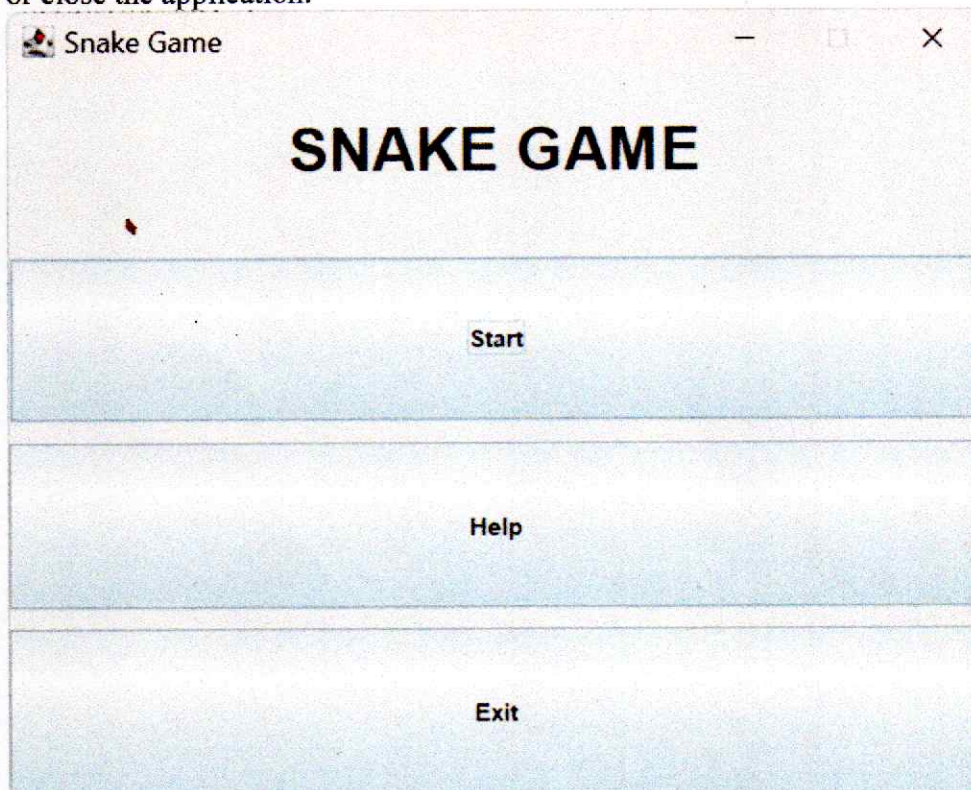
- **Desktop or Laptop Computer** – Required to develop and run the SnakeGame application.
- **Keyboard** – Used to control the snake during gameplay.
- **Monitor/Display** – Required to display the Java Swing game interface.

2. Software Requirements

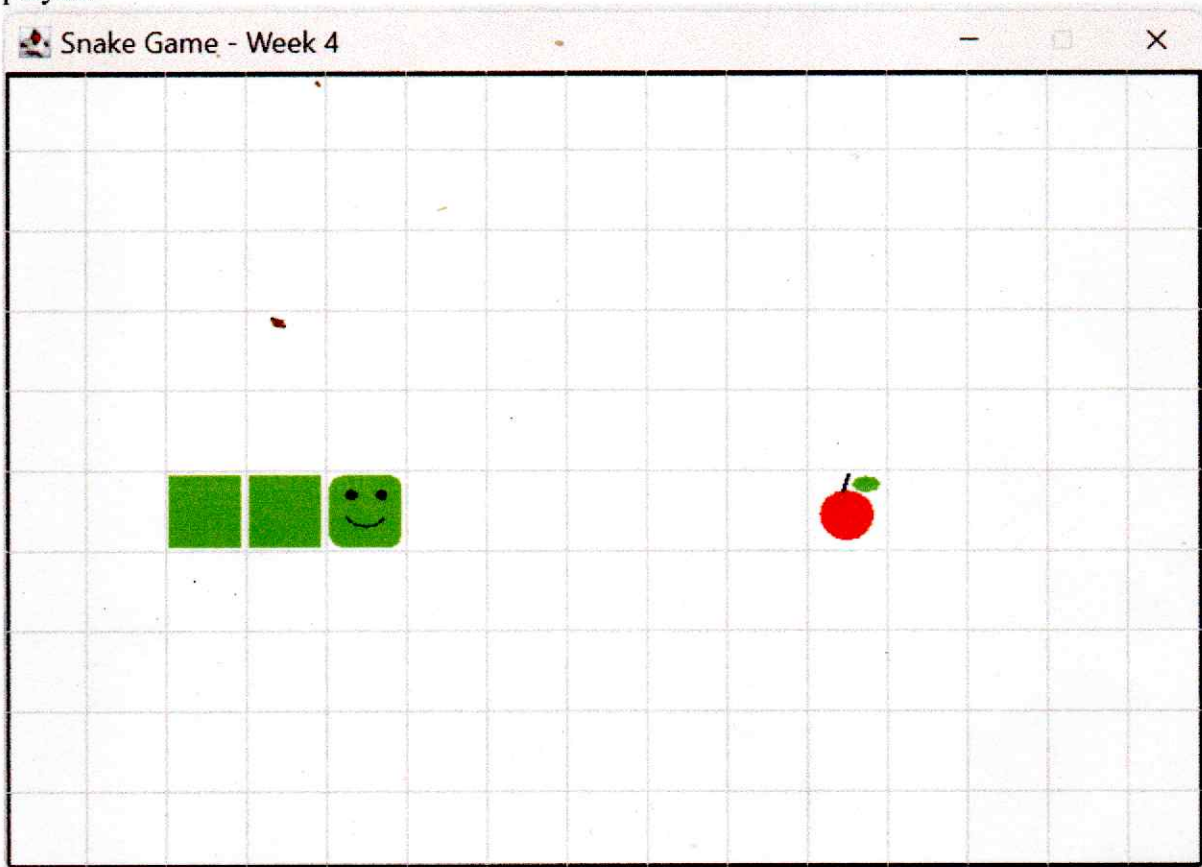
- **Java Development Kit (JDK)** – Required to compile and run the Java program.
- **NetBeans IDE** – Used for writing, running, testing, and debugging the project.
- **Operating System** – Windows, Linux, or another operating system capable of running Java Swing

6. System Features

- The **SnakeGame** system contains several features that are developed step by step throughout the project.
- **6.1 Main Menu**
- The Main Menu provides the basic options for the application. It includes **Start**, **Help**, and **Exit** options. The user can select an option to start the game, view help information, or close the application.

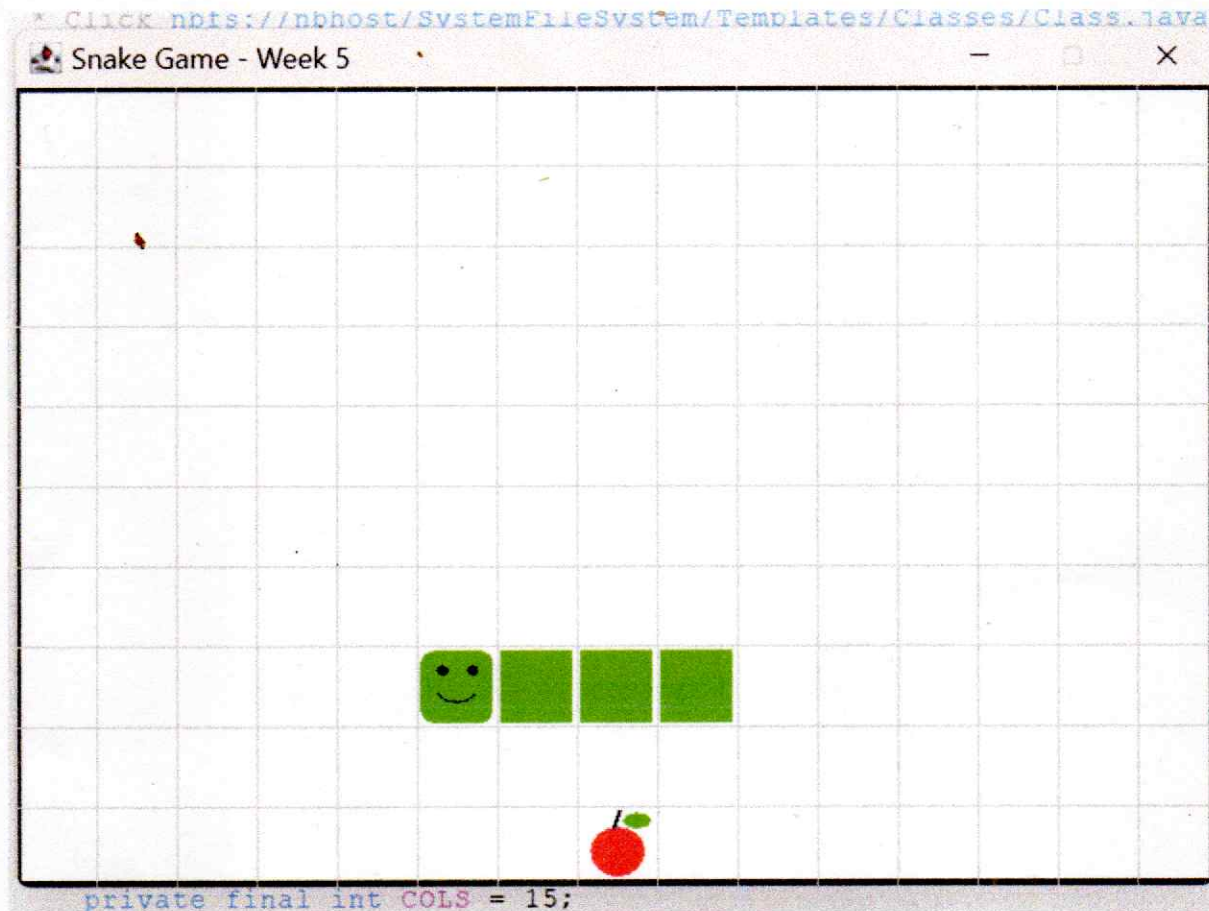


- 6.2 Game Board
- The Game Board provides the main playing area of the SnakeGame. It contains the **grid**, **snake**, and **food**. The board provides a fixed area where the snake moves and the game is played.



- 6.3 Snake Movement

- The snake can move in different directions according to the user's keyboard input. The movement system controls the snake's position on the game board and prevents invalid direct reverse movement.



- 6.4 Food Collection

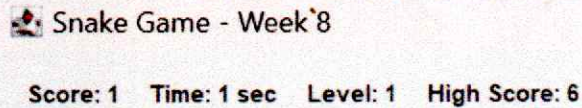
- Food is displayed on the game board for the snake to collect. When the snake's head reaches the food, the food is collected and the **snake becomes longer**. A new food position is then generated.

- 6.5 Collision Detection

- The system detects collisions during gameplay. It checks whether the snake hits the **game-board wall** or its **own body**. When a collision occurs, the game ends and the Game Over condition is shown.

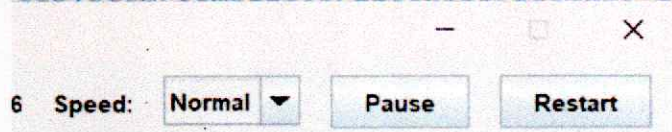
6.6 Score System

- The score system keeps track of the player's progress during the game. The player's score increases as the snake successfully collects food.



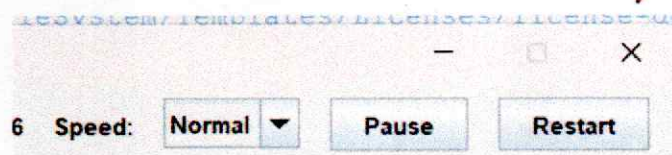
6.7 Time and Speed Control

- The game manages the movement speed of the snake and the game time. This helps control the gameplay and provides different levels of challenge.

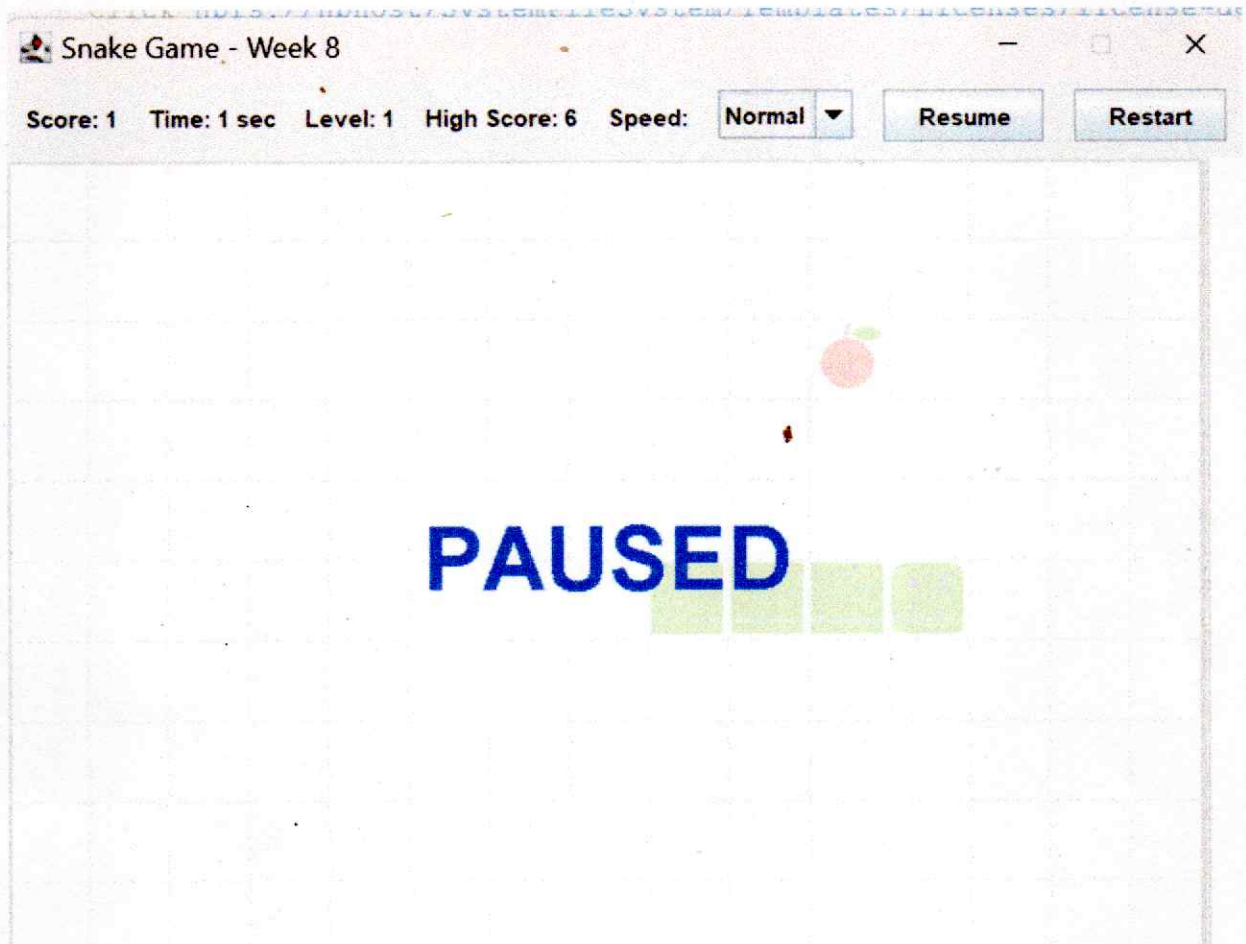


6.8 Restart Function

- The Restart feature allows the player to start the game again after the game ends. It resets the necessary game conditions and starts a new game.

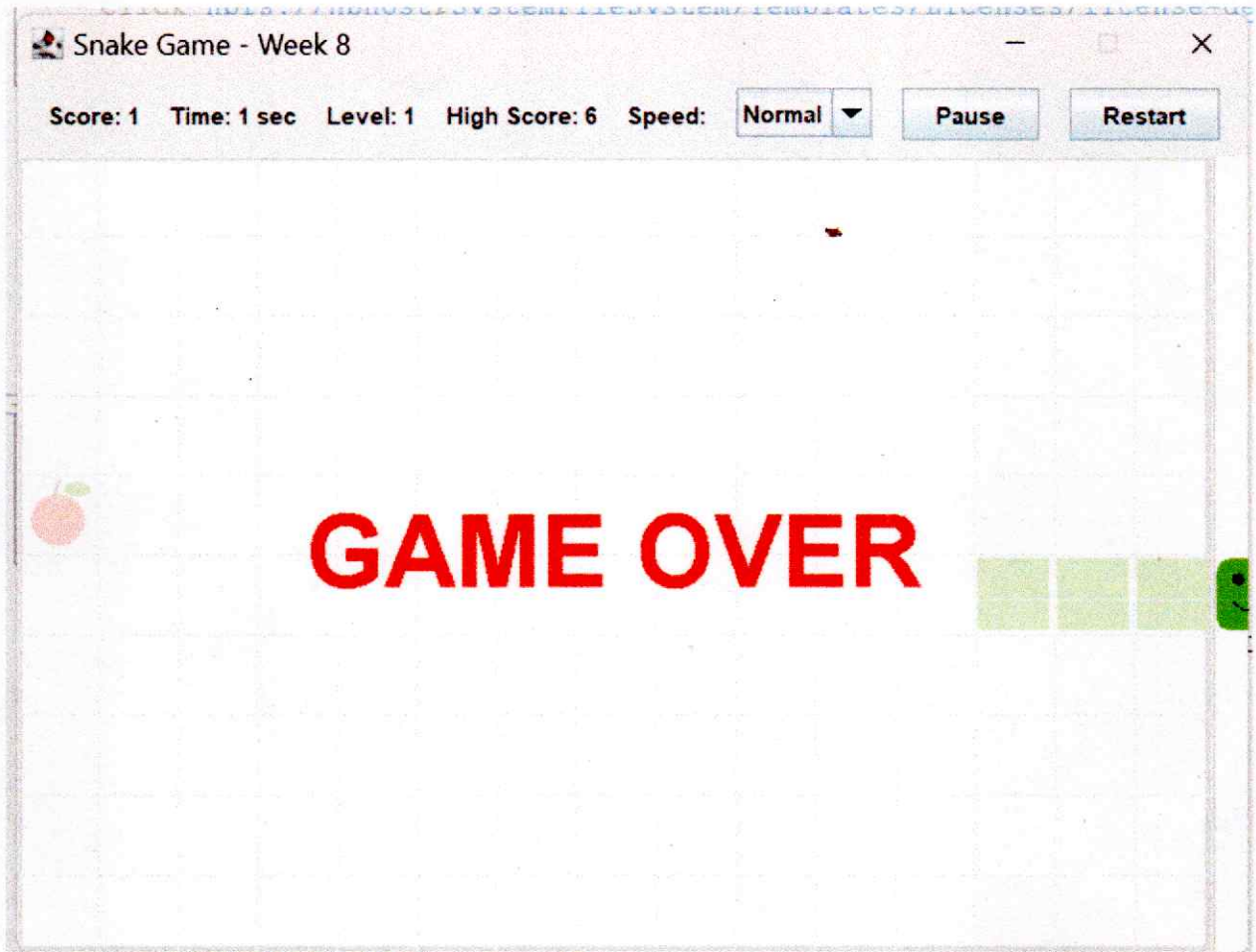


- 6.9 Pause and Resume
- The Pause and Resume feature allows the player to temporarily stop the game and continue it later. This provides control over the current game state.



- 6.10 Different Game Levels
- The system provides different game levels. Each level provides a different stage of gameplay and allows the player to progress through the game.
- 6.11 High-Score System
- The High-Score System stores the player's high score using file storage. This allows the game to keep the player's best score for future reference.

- 6.12 Win/Target Score Detection
- The system checks whether the player reaches the required target score. When the target condition is achieved, the game can identify the successful completion of the game.
-
- 6.13 Game Over and Success Messages
- The system displays appropriate messages according to the game condition. A **Game Over** message is shown when the player loses, while a **Success/Winning** message is used when the required target is achieved.

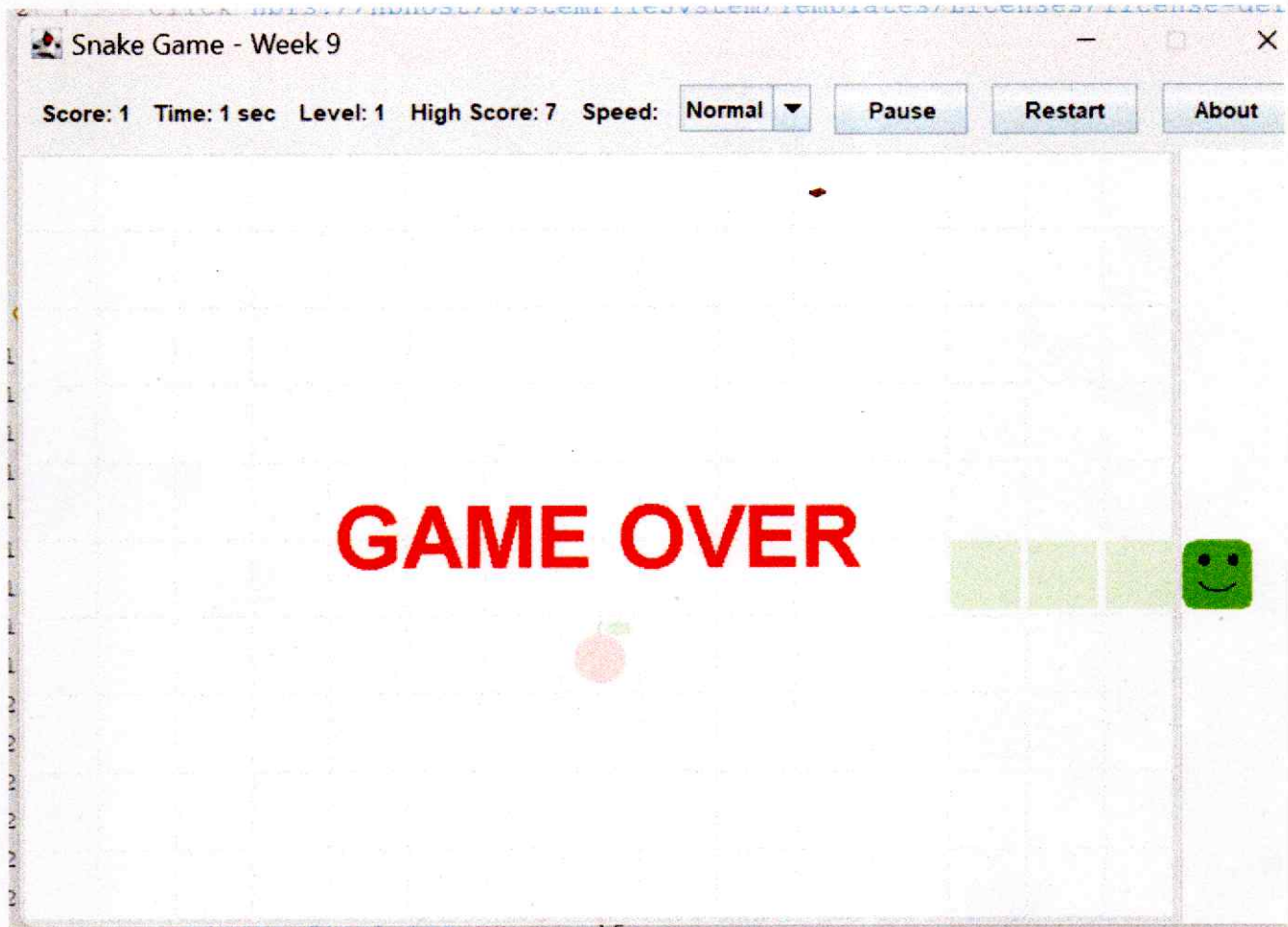


6.14 About Section

- The About section provides information about the SnakeGame application.

6.15 Testing and Final Release

- Before the final release, the complete application is tested. Bugs and errors are identified and fixed. The final source code, documentation, user manual, and screenshots are prepared for the **Version 1.0 release**.



7. System Architecture

The **SnakeGame** system follows a simple **Java Swing-based architecture**. The application is divided into several main parts that work together to run the game.

7.1 User Interface Layer

This layer provides the graphical game interface. It displays the game window, game board, grid, snake, and food. Java Swing components such as `JFrame` and `JPanel` are used to create the interface.

7.2 Game Control Layer

This part controls the main game operations. It manages the game loop, snake movement, food collection, and screen updates. A `Timer` is used to repeatedly update the game.

7.3 Snake and Food Management

The snake is stored using an `ArrayList<Point>`, while the food is represented by a `Point`. The system updates their positions during gameplay.

7.4 Keyboard Control

The system receives keyboard input through `KeyListener`. The **Up, Down, Left, and Right** arrow keys control the snake's movement.

7.5 Game Logic

The game logic controls movement and food collection. When the snake reaches the food, the snake grows and a new food position is generated.

7.6 Collision and Boundary Control

The system checks the snake's position to keep it inside the game board. It also handles the required game conditions during movement.

7.7 Rendering System

The `paintComponent()` method draws the game board, grid, snake, and apple on the screen. The screen is refreshed after game updates.

7.8 Overall Architecture Flow

User Input → Game Control → Snake/Food Update → Game Logic → Rendering → Screen Display

This architecture keeps the main parts of the SnakeGame organized and allows the game to update and display its current state continuously.

Available next action: Create a downloadable DOCX file here in this chat containing the editable prose above

8. Project Structure

The **SnakeGame** project is organized into a Java package and a main Java class.

8.1 Package

Package Name: snakegame

The package contains the main source code of the SnakeGame application.

8.2 Main Class

Class Name: SnakeGame

The **SnakeGame** class contains the main game implementation. It handles the game board, snake, food, movement, keyboard control, game loop, and drawing.

8.3 Main Components

- **Game Settings** – Defines cell size, number of rows and columns, and game-board dimensions.
- **Snake and Food** – Uses `ArrayList<Point>` for the snake and `Point` for the food.
- **Snake Movement** – Controls the snake's movement using direction values.
- **Game Loop** – Uses `Java Timer` to update the game continuously.
- **Food Generation** – Generates a new food position on the board.
- **Game Board Drawing** – Draws the border, grid, snake, and apple.
- **Keyboard Control** – Uses `KeyListener` to control movement with arrow keys.
- **Main Method** – Creates the `JFrame`, adds the game panel, and starts the application.

8.4 Basic Project Structure

```
SnakeGame
├── Source Packages
│   └── snakegame
│       └── SnakeGame.java
```

9. Game Rules

The **SnakeGame** follows these basic rules:

1. **Snake Movement** – The snake moves continuously on the game board and can be controlled using the **Up, Down, Left, and Right arrow keys**.
2. **Food Collection** – The player must move the snake toward the apple. When the snake's head reaches the food, the snake grows longer and a new food position is generated.
3. **Board Boundary** – The snake must remain inside the game board. The movement code checks the board boundaries before moving the snake.
4. **Direction Rule** – The snake cannot immediately reverse its direction. For example, when moving right, it cannot directly move left.
5. **Continuous Movement** – A `Java Timer` controls the game loop and repeatedly updates the snake's movement.
6. **Objective** – The main objective is to control the snake, collect the food, and make the snake grow while continuing to play within the game board.

10. Game Flow

The `SnakeGame` follows this sequence:

Start Game → Initialize Snake & Food → Snake Moves → User Controls Direction → Check Food → Snake Grows → Generate New Food → Continue Game

Step-by-Step

1. **Start Game**
The `SnakeGame` application starts and initializes the game board, snake, food, and movement direction.
2. **Initialize Snake and Food**
The snake is created with its initial body positions, and the first food is placed on the game board.
3. **Start Game Loop**
A `Java Timer` starts the game loop and continuously updates the game.
4. **Snake Movement**
The snake's head position is calculated according to the current direction.
5. **User Controls Direction**
The player uses the **Up, Down, Left, and Right arrow keys** to change the snake's direction.
6. **Check Food**
The system checks whether the snake's new head position matches the food.
7. **Snake Grows**
If the snake eats the food, its tail is not removed, so the snake becomes longer. A new food position is then generated.
8. **Continue Game**
If the snake does not eat the food, its tail is removed and the snake continues moving. The game continues through the game loop.

11. Testing

The **SnakeGame** application is tested to make sure that the implemented features work correctly and the game runs properly.

11.1 Game Launch Testing

The application is run to check whether the SnakeGame window opens correctly without errors.

11.2 Snake Movement Testing

The **Up, Down, Left, and Right** arrow keys are tested to verify that the snake moves in the selected direction.

11.3 Food Collection Testing

The snake is moved toward the apple to check whether the food is detected correctly and the snake becomes longer.

11.4 Food Generation Testing

After the snake eats the food, the system is checked to ensure that a new food position is generated.

11.5 Boundary Testing

The snake's movement near the game-board boundaries is tested to make sure it does not move outside the board.

11.6 Direction Testing

The movement controls are tested to verify that the snake cannot immediately move in the opposite direction.

14.7 Game Loop Testing

The game loop is tested to ensure that the snake continues moving and the screen updates correctly.

11.8 Bug and Error Testing

The application is checked for bugs and errors during gameplay. Any identified problems are fixed before the final project release.

Testing Result: The testing process verifies the main implemented SnakeGame functions, including game initialization, movement, food collection, food generation, boundary control, and keyboard control.

12. Conclusion

The **SnakeGame** project was developed step by step from planning and design to implementation, testing, and final release. The project includes the main game components such as the game board, snake, food, movement control, food collection, and game flow.

Throughout the development process, the project was improved and tested to ensure that the implemented features work properly. The final stage focuses on complete testing, fixing bugs and errors, preparing documentation and presentation, and releasing the final version of the project.

Overall, the **SnakeGame** project provides a complete Java-based game application and demonstrates the practical use of programming, GUI development, event handling, game logic, and software testing.

Available next action: Create a downloadable DOCX file here in this chat containing the editable prose above

13. Project Demonstration

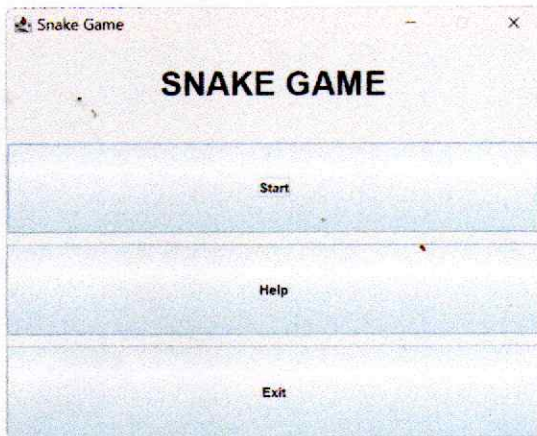
The **SnakeGame** project will be demonstrated as a complete working application.

During the demonstration, the following features will be shown:

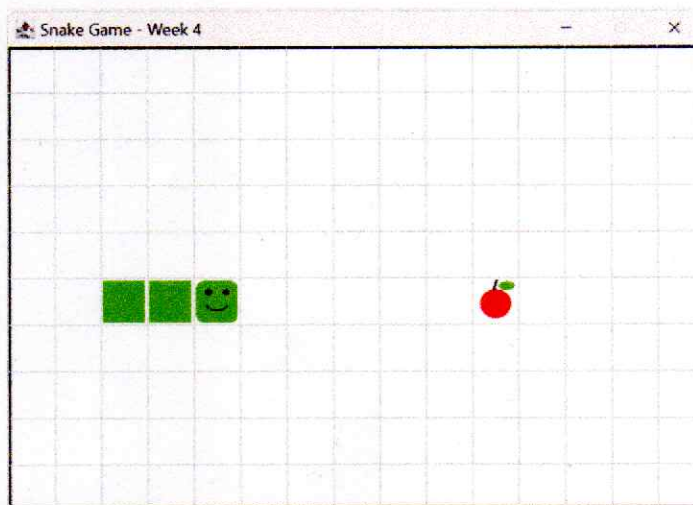
1. **Complete Snake Game Application** – Run the completed SnakeGame application.
2. **Game Board** – Show the game board, grid, snake, and food.
3. **Snake Movement** – Demonstrate movement using the arrow keys.
4. **Food Collection** – Show the snake collecting food and growing.
5. **Game Controls** – Demonstrate the implemented keyboard controls.
6. **Game Features** – Demonstrate the implemented features of the completed project.
7. **All Features Working** – Show that the complete application runs correctly after testing.

Demonstration Screenshots

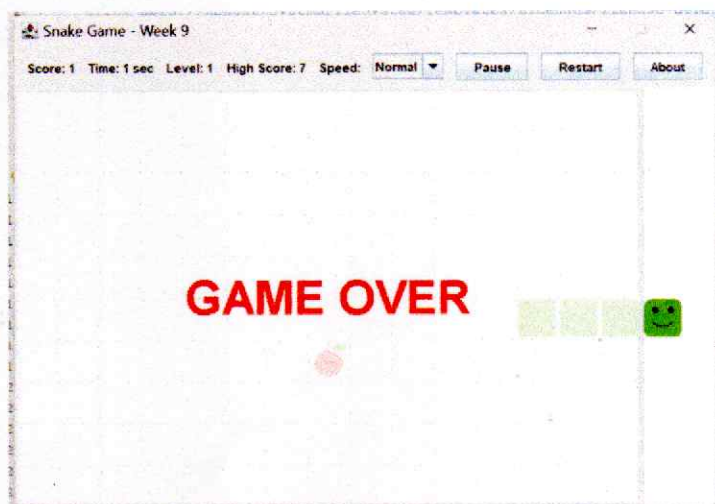
Screenshot 1: Main Game Window



Screenshot 2: Snake Collecting Food



Screenshot 3: Final Working Game



This demonstration will show the **complete SnakeGame application and its working features**.

14. Final Release

The **SnakeGame** project is prepared for final release after completing the required testing and fixing bugs and errors.

The final release includes:

- **Final Source Code**
- **Documentation**
- **User Manual**
- **Screenshots**
- **Version 1.0 Release**

The final project is prepared as a **Version 1.0 Release** after completing the testing and documentation work.